LAPORAN PRAKTIKUM STRUKTUR DATA

PERTEMUAN 1

OLEH:

LEXI MULIA YUNASPI

2511531006

MATA KULIAH STRUKTUR DATA
DOSEN PENGAMPU : Dr, WAHYUDI ,M.T

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data yang berjudul *Implementasi Kelas Jam dalam Pemrograman Berorientasi Objek* dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum yang bertujuan untuk memahami dan mengimplementasikan konsep pemrograman berorientasi objek, khususnya dalam pembuatan kelas dan objek yang merepresentasikan waktu. Melalui praktikum ini, penulis mempelajari bagaimana melakukan validasi data, konversi waktu, perbandingan, serta operasi aritmatika terhadap objek jam.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penyajian maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang bersifat membangun demi penyempurnaan laporan ini di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi penulis khususnya dan pembaca pada umumnya.

Padang 02 April

Penulis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN iii

I.I Latar Belakang 1

1.2 Tujuan .....,,,,,,,,,,,,,,,,,,,,,,,.....1

1.3 Manfaat .....,,,.....2

BAB II PEMBAHASAN.....,3

2.2 Langkah Langkah praktikum ……...…3

2.3 Langkah Langkah tugas praktikum 21

BAB III PENUTUP ……..22

3.1 Kesimpulan …….....… ...22

3.2 Saran ……..…..22

DAFTAR PUSTAKA 23

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan teknologi informasi, pemrograman berorientasi objek merupakan salah satu paradigma yang banyak digunakan dalam pengembangan perangkat lunak. Paradigma ini memungkinkan pengembang untuk merepresentasikan objek-objek nyata ke dalam bentuk program secara lebih terstruktur dan sistematis.

Salah satu konsep yang dapat direpresentasikan dalam pemrograman adalah waktu, yang terdiri atas jam, menit, dan detik. Pengolahan waktu dalam suatu program memerlukan ketelitian karena adanya batasan nilai serta berbagai operasi yang dapat dilakukan, seperti konversi, perhitungan durasi, dan perbandingan waktu.

Oleh karena itu, diperlukan suatu struktur data yang mampu merepresentasikan waktu secara tepat dan menyediakan berbagai operasi yang diperlukan. Melalui praktikum ini, dikembangkan sebuah kelas "Jam" yang mengimplementasikan konsep tersebut dalam bahasa pemrograman Java, sehingga dapat membantu mahasiswa dalam memahami penerapan konsep abstraksi dan enkapsulasi dalam pemrograman.

1.2 Tujuan

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

A. Memahami konsep dasar pemrograman berorientasi objek.
B. Mampu merancang dan mengimplementasikan kelas serta objek dalam bahasa pemrograman Java.
C. Memahami penggunaan atribut dan method dalam suatu kelas.
D. Mampu melakukan validasi data pada objek yang dibuat.
E. Memahami penerapan operasi aritmatika dan relasional pada objek waktu.

F. Mampu mengembangkan program yang bersifat interaktif melalui input pengguna.

1.3 Manfaat

Manfaat yang diperoleh dari pelaksanaan praktikum ini adalah sebagai berikut:

A. Meningkatkan pemahaman mahasiswa terhadap konsep pemrograman berorientasi objek.
B. Memberikan pengalaman dalam merancang dan mengimplementasikan kelas serta objek dalam program.
C. Melatih kemampuan dalam melakukan validasi dan pengolahan data secara sistematis.
D. Mengembangkan kemampuan logika dalam menyelesaikan permasalahan yang berkaitan dengan waktu.
E. Meningkatkan keterampilan dalam membuat program yang terstruktur dan interaktif.

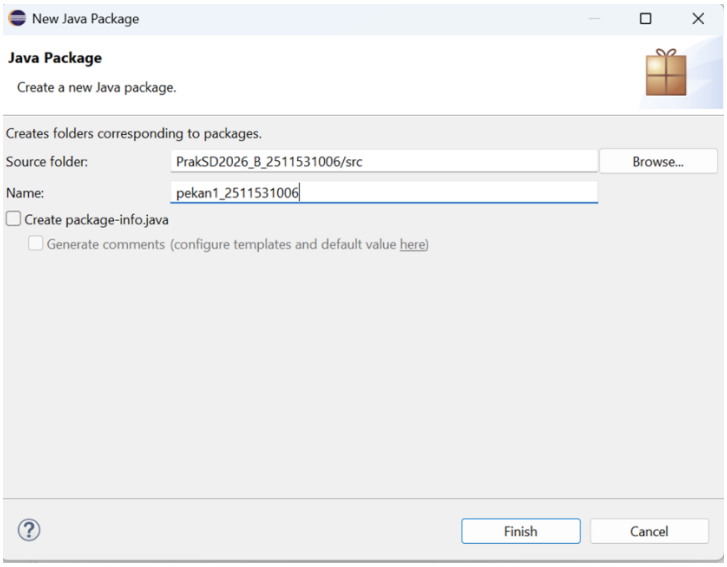
BAB II

PEMBAHASAN

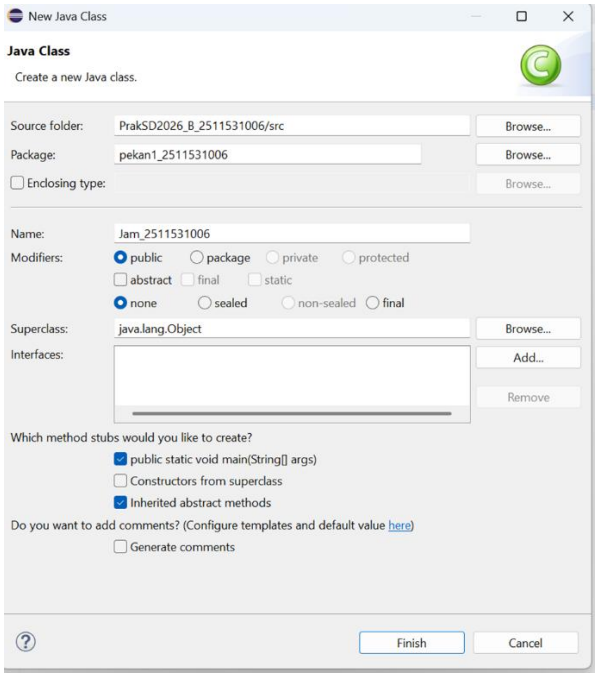
1.1 Langkah Langkah Praktikum

1. Program 1

A. Tampilan tersebut menunjukkan proses pembuatan *package* baru pada lingkungan pengembangan Java, yaitu Eclipse IDE. Pada bagian *Name*, pengguna memasukkan nama *package* yaitu pekan1_2511531006, yang berfungsi sebagai wadah untuk mengelompokkan kelas-kelas program agar lebih terstruktur. Setelah pengisian selesai, pengguna dapat menekan tombol Finish untuk membuat *package* tersebut di dalam folder sumber (*src*).



B. Tampilan tersebut menunjukkan proses pembuatan kelas baru pada Eclipse IDE. Pada bagian *Package*, kelas ditempatkan dalam *package* pekan1_2511531006, sedangkan pada bagian *Name* diisi dengan nama kelas yaitu Jam_2511531006. Kelas ini dibuat dengan modifier public, sehingga dapat diakses dari kelas lain. Selain itu, opsi pembuatan method main dipilih untuk memungkinkan kelas dijalankan sebagai program utama. Setelah seluruh pengaturan selesai, pengguna dapat menekan tombol Finish untuk membuat kelas tersebut.



C. Bagian ini menunjukkan bahwa kelas Jam_2511531006 berada dalam *package* pekan1_2511531006. Kelas dideklarasikan dengan modifier public final**,** yang berarti kelas dapat diakses dari luar *package* dan tidak dapat diturunkan (inheritance) oleh kelas lain.

```
1 package pekan1_2511531006;
2
3 public final class Jam_2511531006 {
```

D. Bagian kode tersebut menunjukkan deklarasi atribut dalam kelas Jam_2511531006. Terdapat tiga variabel bertipe data int, yaitu hh untuk menyimpan nilai jam dengan rentang 0–23, mm untuk menyimpan nilai menit dengan rentang 0–59, dan ss untuk menyimpan nilai detik dengan rentang 0–59. Ketiga atribut tersebut diberi modifier **private**, yang berarti hanya dapat diakses langsung di dalam kelas, sehingga menerapkan prinsip enkapsulasi dalam pemrograman berorientasi objek.

```
4     private int hh; // 0..23
5     private int mm; // 0..59
6     private int ss; // 0..59
7
```

E. Kode tersebut adalah sebuah method bernama isValid yang berfungsi untuk memvalidasi nilai waktu (jam, menit, dan detik). Method ini menerima tiga parameter integer yaitu h (jam), m (menit), dan s (detik), lalu mengembalikan nilai boolean (true atau false). Nilai akan dianggap valid (true) jika jam berada di antara 0–23, serta menit dan detik berada di antara 0–59. Jika salah satu nilai berada di luar batas tersebut, maka method akan mengembalikan false.

```
7
8    // ---Validator---
9    public static boolean isValid(int h, int m, int s) {
10       return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
11   }
12
```

F. Kode tersebut merupakan konstruktor (constructor) dari kelas Jam_2511531006 yang digunakan untuk menginisialisasi objek saat pertama kali dibuat. Konstruktor ini menerima tiga parameter yaitu h (jam), m (menit), dan s (detik), lalu menyimpan nilai tersebut ke dalam atribut objek hh, mm, dan ss menggunakan keyword this. Dengan demikian, setiap objek yang dibuat dari kelas ini akan langsung memiliki nilai waktu sesuai dengan input yang diberikan.

```
12
13      //---Konstruktor/Kreator---
14      public Jam_2511531006(int h, int m, int s) {
15          this.hh = h; this.mm = m; this.ss = s;
16      }
```

G. Kode tersebut merupakan method selector (getter) yang digunakan untuk mengambil nilai atribut dari objek tanpa mengubahnya. Terdapat tiga method yaitu getHour(), getMinute(), dan getSecond() yang masing-masing mengembalikan nilai hh (jam), mm (menit), dan ss (detik). Method ini berguna agar atribut tetap bersifat enkapsulasi (tidak diakses langsung dari luar), tetapi nilainya tetap bisa dibaca melalui fungsi yang disediakan.

```
18      //---Selector---
19      public int getHour() { return hh; }
20      public int getMinute() { return mm; }
21      public int getSecond() { return ss; }
22
```

H. Kode tersebut merupakan method mutator (setter) yang digunakan untuk mengubah nilai atribut dalam objek. Terdapat tiga method yaitu setHour(int h), setMinute(int m), dan setSecond(int s) yang masing-masing berfungsi untuk mengganti nilai hh (jam), mm (menit), dan ss (detik) dengan nilai baru yang diberikan. Penggunaan this menunjukkan bahwa nilai parameter akan disimpan ke atribut milik objek, sehingga data waktu dapat diperbarui setelah objek dibuat.

```
    //---Mutator---
    public void setHour(int h) {this.hh = h; }
    public void setMinute(int m) {this.mm = m; }
    public void setSecond(int s) {this.ss = s; }

```

I. Kode tersebut merupakan bagian konversi waktu. Method toSeconds() digunakan untuk mengubah waktu (jam, menit, detik) menjadi total detik dengan rumus hh * 3600 + mm * 60 + ss. Sedangkan method fromSeconds(int total) berfungsi untuk mengubah total detik menjadi objek waktu (Jam_2511531006). Method ini juga melakukan validasi jika nilai detik negatif, lalu menggunakan operasi modulo untuk membatasi dalam 24 jam, kemudian menghitung kembali jam, menit, dan detik sebelum akhirnya mengembalikan objek baru dengan nilai tersebut.

```
    //---Konversi---
    public int  toSeconds() { return hh * 3600 + mm * 60 * ss; }
    public static Jam_2511531006 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600; //wrap 24 jam
        int h = total/ 3600; total %= 3600;
        int m = total/ 60; int s = total % 60;
        return new Jam_2511531006(h, m, s);
    }
```

J. Kode tersebut merupakan bagian relasional yang digunakan untuk membandingkan objek waktu. Method compareTo() membandingkan dua objek Jam berdasarkan total detik menggunakan Integer.compare, sehingga bisa diketahui urutan waktu (lebih kecil, sama, atau lebih besar). Method equals() digunakan untuk mengecek apakah dua objek memiliki nilai jam, menit, dan detik yang sama, dengan terlebih dahulu memastikan objek yang dibandingkan bertipe sama. Sedangkan hashCode() menghasilkan kode hash berdasarkan nilai hh, mm, dan ss, yang berguna saat objek digunakan dalam struktur data seperti HashMap atau HashSet agar konsisten dengan equals().

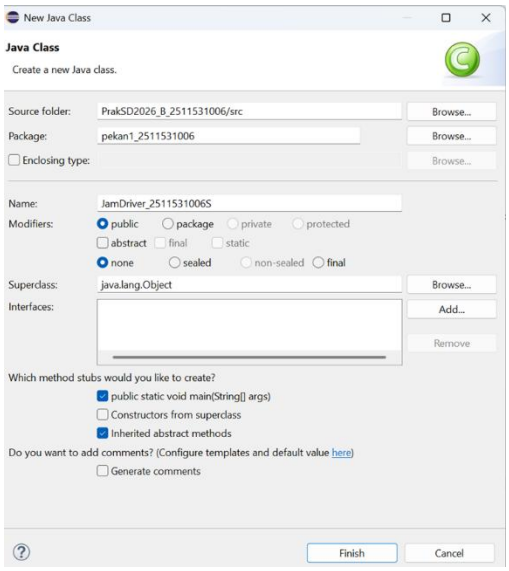
```java
    //---Relasional---
    public int compareTo(Jam_2511531006 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (! (o instanceof Jam_2511531006 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
        public int hashCode() { return java.util.Objects.hash(hh, mm, ss);}
```

K. Kode tersebut merupakan bagian aritmatika waktu yang menyediakan operasi penjumlahan, pengurangan, dan manipulasi detik pada objek Jam. Method plus() dan minus() digunakan untuk menjumlahkan atau mengurangkan dua waktu dengan mengonversinya ke detik terlebih dahulu. Method nextSecond() dan previousSecond() berfungsi untuk menambah atau mengurangi satu detik, sedangkan versi dengan parameter n memungkinkan penambahan atau pengurangan beberapa detik sekaligus dengan tetap menjaga hasil dalam rentang 24 jam menggunakan Math.floorMod. Method durasiDetik() menghitung selisih waktu dalam detik antara dua objek (bisa bernilai negatif). Terakhir, toString() digunakan untuk menampilkan waktu dalam format hh:mm:ss dengan dua digit setiap bagian.

```java
    //---Aritmatika---
     public Jam_2511531006 plus(Jam_2511531006 other) {return fromSeconds(this.toSeconds() + other.toSeconds()); }
     public Jam_2511531006 minus(Jam_2511531006 other) {return fromSeconds(this.toSeconds() - other.toSeconds());}
     public Jam_2511531006 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
     public Jam_2511531006 nextNextSecond(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n));}
     public Jam_2511531006 previousSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600));   }
     public Jam_2511531006 previousNSecond(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24

     //---Durasi (detik) -- bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAkh)
     public static int durasiDetik(Jam_2511531006 jAw, Jam_2511531006 jAkh) { return jAkh.toSeconds() - jAw.toSeconds();}

     //override toString
     public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss);}


    }
```

2. Program 2

A. Gambar tersebut menunjukkan jendela pembuatan kelas baru di Java (New Java Class) pada IDE. Pada bagian ini, pengguna menentukan lokasi folder sumber (src), nama package (pekan1_2511531006), serta nama kelas yaitu JamDriver_2511531006. Kelas diset sebagai public agar dapat diakses dari luar package. Selain itu, terdapat opsi untuk langsung membuat method main() yang berfungsi sebagai titik awal program dijalankan. Dengan pengaturan ini, kelas yang dibuat akan digunakan sebagai driver atau program utama untuk menjalankan dan menguji class Jam.



B. Kode tersebut merupakan bagian awal dari file Java yang mendefinisikan package dan deklarasi kelas. Baris package pekan1_2511531006; menunjukkan bahwa kelas ini berada dalam package tersebut. Kemudian, public class JamDriver_2511531006 { adalah deklarasi kelas utama yang bersifat public, sehingga dapat diakses dari luar package. Kelas ini biasanya digunakan sebagai driver atau program utama untuk menjalankan dan menguji fungsi dari class Jam.

```
1  package pekan1_2511531006;
2
3  public class JamDriver_2511531006 {
4
```

C. Kode tersebut merupakan bagian dari method main() yang menjadi titik awal eksekusi program Java. Di dalamnya dibuat dua objek dari kelas Jam_2511531006, yaitu a dan b. Objek a diinisialisasi dengan waktu 23 jam 59 menit 50 detik, sedangkan objek b dengan waktu 00 jam 00 menit 15 detik. Pembuatan objek ini bertujuan untuk menguji fungsi-fungsi yang ada pada kelas Jam, seperti operasi waktu atau perbandingan.

```
5⊖     public static void main(String[] args) {
6          Jam_2511531006 a = new Jam_2511531006(23,59,50);
7          Jam_2511531006 b = new Jam_2511531006(0,0,15);
```

D. Kode tersebut digunakan untuk menampilkan hasil operasi objek Jam ke layar menggunakan System.out.println. Baris pertama dan kedua mencetak nilai objek a dan b. Baris berikutnya menampilkan hasil penjumlahan waktu a.plus(b). Kemudian, durasiDetik(a,b) digunakan untuk menghitung selisih waktu dalam detik antara a dan b. Terakhir, a.compareTo(b) menampilkan hasil perbandingan antara kedua waktu tersebut, apakah a lebih kecil, sama, atau lebih besar dari b.

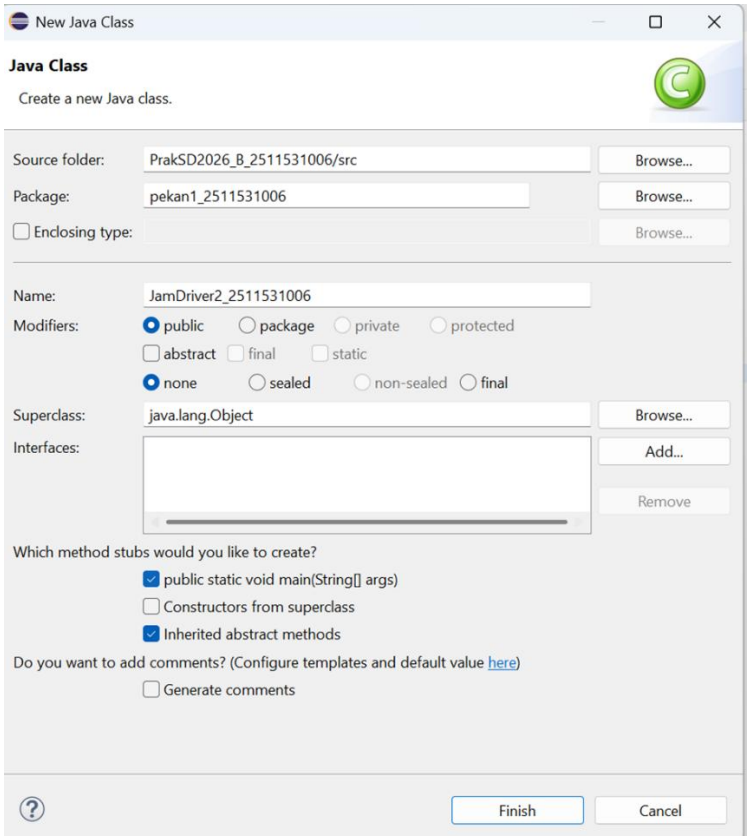
```
        System.out.println("a        =" + a);
        System.out.println("b        =" + b);
        System.out.println("ab       =" + a.plus(b));
        System.out.println("next 20s =" + Jam_2511531006.durasiDetik(a,b));
        System.out.println("a.compareTo(b) =" + a.compareTo(b));


    }

}
```

E. Output

```
Console
<terminated> JamDriver_2511531006 (1) [Java Application] C:\Users\Hyp
a         =23:59:50
b         =00:00:15
ab        =00:10:00
next 20s =-259800
a.compareTo(b) =1
```

3. Program 3

A. Gambar tersebut menunjukkan proses pembuatan kelas Java baru bernama JamDriver2_2511531006 di dalam package pekan1_2511531006. Kelas ini diset sebagai public agar dapat diakses secara luas, dan opsi public static void main(String[] args) dicentang sehingga otomatis dibuat method main() sebagai titik awal program. Kelas ini biasanya digunakan sebagai driver kedua untuk menguji atau menjalankan fungsi tambahan dari class Jam dengan skenario yang berbeda dari driver sebelumnya.



B. Kode tersebut merupakan bagian awal dari program Java yang mendefinisikan package, import, dan deklarasi kelas. Baris package pekan1_2511531006; menunjukkan lokasi kelas dalam package. Kemudian import java.util.Scanner; digunakan untuk mengimpor kelas Scanner agar program dapat menerima input dari pengguna. Selanjutnya, public class JamDriver2_2511531006 adalah deklarasi kelas utama yang bersifat public, dan di dalamnya terdapat method main() sebagai titik awal eksekusi program.

```
1  package pekan1_2511531006;
2
3  import java.util.Scanner;
4
5  public class JamDriver2_2511531006 {
6
7      public static void main(String[] args) {
```

C. Kode tersebut digunakan untuk memulai interaksi dengan pengguna. Baris Scanner input = new Scanner(System.in); berfungsi untuk membuat objek Scanner agar program bisa menerima input dari keyboard. Kemudian

System.out.println("=== Program Driver Objek Jam ==="); menampilkan judul program. Selanjutnya ditampilkan pesan Input Jam 1 sebagai petunjuk, dan method buatJamDariInput(input) dipanggil untuk membaca input dari pengguna lalu membuat objek Jam pertama (j1) berdasarkan data yang dimasukkan.

```
8        Scanner input = new Scanner(System.in);
9        System.out.println("=== Program Driver Objek Jam ===");
10       // 1. Input Jam Pertama
11       System.out.println("\n--- Input Jam 1 ---");
12       Jam_2511531006 j1 = buatJamDariInput(input);
```

D. Gambar tersebut tampak buram dan terbalik, sehingga teks di dalamnya tidak dapat terbaca dengan jelas. Kemungkinan itu adalah potongan tulisan atau dokumen, tetapi perlu diperjelas atau diputar agar bisa dijelaskan dengan tepat. Silakan kirim ulang gambar yang lebih jelas atau dalam posisi yang benar.

```
13
14       // 2. Input Jam Kedua
15       System.out.println("\n--- Input Jam 2 ---");
16       Jam_2511531006 j2 = buatJamDariInput(input);
17
```

E. Kode tersebut berfungsi untuk menampilkan hasil data dan konversi waktu ke layar. Pertama, program mencetak judul “Hasil Operasi”. Kemudian, j1.toString() dan j2.toString() digunakan untuk menampilkan waktu Jam 1 dan Jam 2 dalam format hh:mm:ss. Selanjutnya, j1.toSeconds() dan j2.toSeconds() menampilkan masing-masing waktu dalam bentuk total detik, sehingga memudahkan untuk melihat nilai waktu dalam satuan yang sama.

```
18       // 3. Menampilkan Data
19       System.out.println("\n--- Hasil Operasi ---");
20       System.out.println("Jam 1 (String): " + j1.toString());
21       System.out.println("Jam 2 (String): " + j2.toString());
22       System.out.println("Jam 1 dalam detik: " + j1.toSeconds());
23       System.out.println("Jam 2 dalam detik: " + j2.toSeconds());
```

F. Kode tersebut melakukan perbandingan dua objek waktu (j1 dan j2) menggunakan method `compareTo()`. Hasil perbandingan disimpan dalam

variabel perbandingan. Jika nilainya lebih dari 0**,** berarti `j1` lebih lambat (setelah j2), sehingga mencetak pesan bahwa Jam 1 lebih lambat. Jika kurang dari 0**,** berarti j1 lebih awal (sebelum j2). Sedangkan jika nilainya 0**,** artinya kedua waktu tersebut sama persis.

```
        // 4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status: Jam 1 lebih lambat (setelah Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status: Jam 1 lebih awal (sebelum Jam 2");
        } else {
            System.out.println("Status: Jam 1 dan Jam 2 sama persis");
        }
```

G. Kode tersebut melakukan operasi aritmatika pada objek waktu. Pertama, program menghitung durasi dalam detik antara j1 dan j2 menggunakan method durasiDetik(), lalu menampilkannya. Selanjutnya, j1.nextSecond() digunakan untuk mendapatkan waktu satu detik setelah j1, dan hasilnya disimpan di jNext. Terakhir, j2.previousSecond() digunakan untuk mendapatkan waktu satu detik sebelum j2, yang disimpan di jPrev. Semua hasil tersebut kemudian dicetak ke layar.

```
        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2) : " + Jam_2511531006.durasiDetik(j1, j2) + " detik");

        Jam_2511531006 jNext = j1.nextSecond();
        System.out.println("Jam 1 detik berikutnya: " + jNext);

        Jam_2511531006 jPrev = j2.previousSecond();
        System.out.println("Jam 1 detik sebelumnya: " + jPrev);
```

H. Kode tersebut melakukan penjumlahan dua objek waktu (**j1** dan **j2**) menggunakan method plus(), lalu menyimpan hasilnya ke dalam variabel jHasilPlus dan menampilkannya. Setelah itu, program menutup input dengan input.close() dan mencetak pesan bahwa program telah selesai dijalankan.

```
        // 6. Operasi Penjumlahan Jam
        Jam_2511531006 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2: " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai");
    }
```

I. Kode tersebut adalah method pembantu buatJamDariInput yang digunakan untuk mengambil dan memvalidasi input waktu dari pengguna**.** Program meminta pengguna memasukkan jam, menit, dan detik, lalu memeriksa validitasnya menggunakan method isValid(h, m, s). Jika input valid, method langsung mengembalikan objek Jam_2511531006 baru. Jika tidak valid, program menampilkan pesan error dan mengulang input karena berada dalam perulangan while (true), sehingga proses akan terus berjalan sampai pengguna memasukkan nilai yang benar.

```java
    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
     * sampai user memasukan angka yang benar (0..23, 0..59).
     */
     private static Jam_2511531006 buatJamDariInput(Scanner sc) {

       int h, m, s;
       while (true) {
           System.out.print("Masukan Jam (0-23): ");
           h = sc.nextInt();
           System.out.print("Masukan Menit (0-59): ");
           m = sc.nextInt();
           System.out.print("Masukan Detik (0-59): ");
           s = sc.nextInt();

           // Memanggil method static isValid dari kelas jam
           if (Jam_2511531006.isValid(h, m, s)) {
               return new Jam_2511531006 (h, m, s);
           } else {
               System.out.println("[Error] Input tidak valid! Silahkan ulangi.\n");
           }

       }

    }

}
```

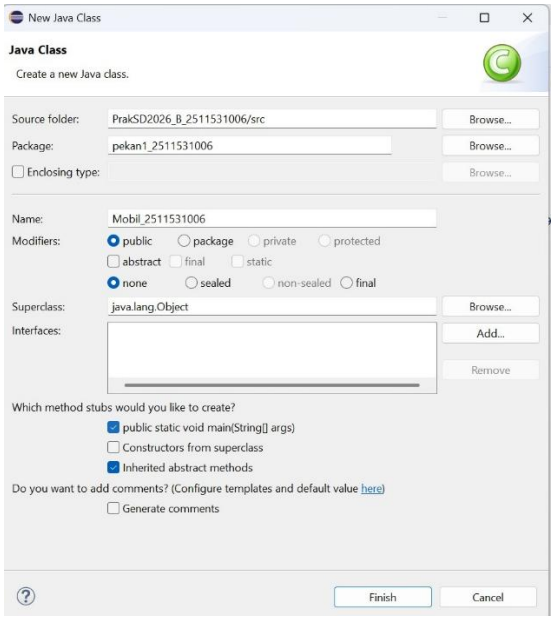
F. Output

```
Console
<terminated> JamDriver2_2511531006 [Java Application]
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukan Jam (0-23): 18
Masukan Menit (0-59): 43
Masukan Detik (0-59): 22

--- Input Jam 2 ---
Masukan Jam (0-23): 17
Masukan Menit (0-59): 50
Masukan Detik (0-59): 47

--- Hasil Operasi ---
Jam 1 (String): 18:43:22
Jam 2 (String): 17:50:47
Jam 1 dalam detik: 121560
Jam 2 dalam detik: 202200
Status: Jam 1 lebih awal (sebelum Jam 2
Durasi (J1 ke J2) : 80640 detik
Jam 1 detik berikutnya: 09:46:01
Jam 1 detik sebelumnya: 08:09:59
Hasil J1 + J2: 17:56:00

Program Selesai
```

1.2 Langkah langkah Tugas Praktikum

1.Program 1

A. Gambar tersebut menampilkan antarmuka pembuatan Java Class baru di IDE Eclipse. Pengguna sedang membuat kelas bernama Mobil_2511531006 yang ditempatkan dalam paket pekan1_2511531006. Konfigurasi menunjukkan bahwa kelas ini bersifat public dan menyertakan metode main, sehingga kelas ini nantinya dapat dieksekusi sebagai program utama



B. Gambar tersebut menunjukkan baris awal kode sumber Java yang dihasilkan dari pengaturan sebelumnya. Baris pertama mendefinisikan package sebagai identitas folder/ruang lingkup kerja, sedangkan baris ketiga adalah deklarasi public class yang menandai dimulainya definisi kelas Mobil_2511531006. Kurung kurawal buka { menandakan bahwa isi atau badan dari kelas tersebut akan dituliskan di bawahnya.

```
1 package pekan1_2511531006;
2
3 public class Mobil_2511531006 {
```

C. Gambar tersebut menunjukkan deklarasi atribut atau variabel anggota di dalam kelas Java. Atribut ini menggunakan modifier akses private, yang berarti data tersebut hanya bisa diakses langsung dari dalam kelas itu sendiri (menerapkan konsep *encapsulation*).

```
4       private String nama;
5       private int tahun;
6       private int cc;
7       private long harga;
8       private String merk;
9
```

D. Gambar tersebut menunjukkan bagian Konstruktor dari kelas Java. Konstruktor ini berfungsi untuk memberikan nilai awal (*inisialisasi*) pada atribut objek saat objek tersebut dibuat.Penggunaan kata kunci this di sini sangat krusial karena nama parameter (variabel di dalam kurung) sama persis dengan nama atribut kelas. Dengan menuliskan this.nama = nama, program dapat membedakan bahwa this.nama merujuk pada atribut milik objek, sedangkan nama merujuk pada data yang dikirimkan melalui parameter, sehingga data tidak bernilai kosong atau null.

```
10      // Konstruktor
11      public Mobil_2511531006(String nama, int tahun, int cc, long harga, String merk) {
12          this.nama = nama;
13          this.tahun = tahun;
14          this.cc = cc;
15          this.harga = harga;
16          this.merk = merk;
17      }
```

E. Gambar tersebut menunjukkan implementasi metode Selektor atau yang lebih umum dikenal sebagai Getter.Metode ini bersifat public dan berfungsi untuk mengambil atau membaca nilai dari atribut yang bersifat private (nama dan merk). Karena atribut tersebut tidak bisa diakses langsung dari luar kelas, metode getNama() dan getMerk() digunakan sebagai "pintu" resmi agar kelas lain dapat memperoleh informasi mengenai nama dan merk mobil tersebut melalui kata kunci return.

```
19      // Selektor (Getter)
20      public String getNama() { return nama; }
21      public String getMerk() { return merk; }
```

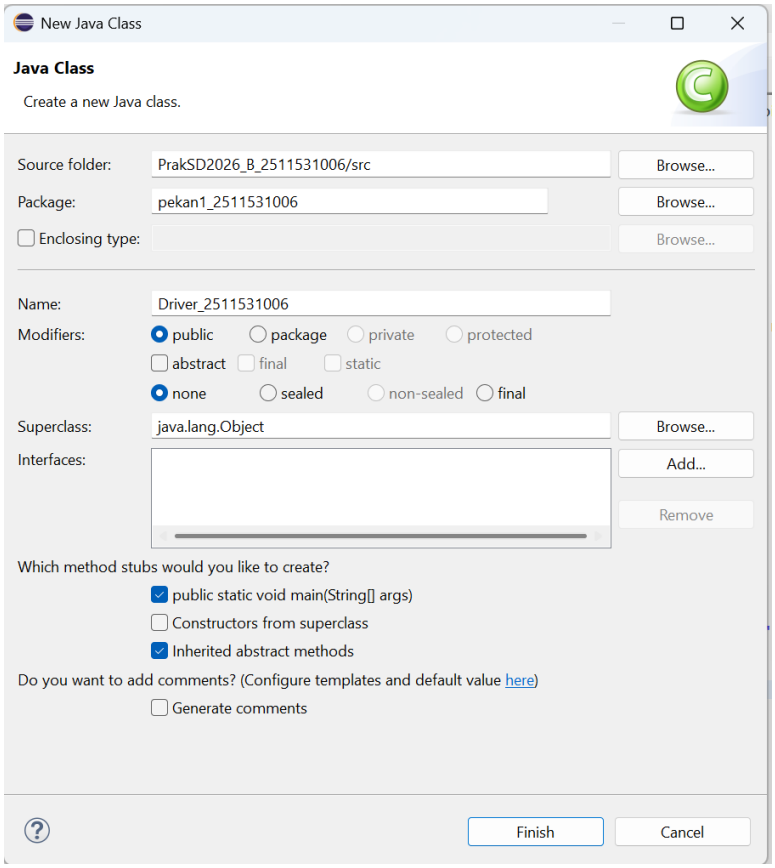
F. Gambar tersebut menunjukkan dua jenis metode dalam kelas Java. Pertama adalah Mutator (Setter) melalui metode setNama, yang berfungsi untuk

mengubah atau memperbarui nilai atribut nama secara terkontrol dari luar kelas menggunakan kata kunci this. Kedua adalah metode displayInfo, yang merupakan metode utilitas untuk menampilkan seluruh informasi objek (nama, merk, tahun, cc, dan harga) ke konsol dalam satu baris format teks yang rapi. Secara keseluruhan, bagian kode ini melengkapi fungsionalitas kelas agar data di dalamnya dapat dimanipulasi dan disajikan dengan mudah.

```java
    // Mutator (Setter)
    public void setNama(String nama) { this.nama = nama; }

    public void displayInfo() {
        System.out.println(nama + " [" + merk + "] - Thn: " + tahun + " - CC: " + cc + " - Harga: " + harga);
    }
}
```

1. Program 2

   A. Gambar ini menunjukkan proses pembuatan kelas Java baru bernama Driver_2511531006 di dalam paket pekan1_2511531006 menggunakan IDE Eclipse. Berbeda dengan kelas model sebelumnya, kelas ini difungsikan sebagai kelas Driver (pengendali) karena memiliki metode public static void main(String[] args) yang telah dicentang. Kelas ini nantinya akan digunakan untuk membuat objek dari kelas Mobil dan menjalankan logika program utama di dalamnya.



B. Gambar tersebut menunjukkan struktur awal dari file kode Java untuk kelas Driver_2511531006. Pada baris pertama, terdapat deklarasi package yang menempatkan kelas ini ke dalam paket bernama pekan1_2511531006 guna mengorganisir file proyek. Di baris ketiga, kata kunci public class mendefinisikan kelas utama dengan akses terbuka, di mana kurung kurawal buka { menandakan dimulainya blok kode yang akan berisi logika utama program atau pembuatan objek dari kelas lain.

```
1 package pekan1_2511531006;
2
3 public class Driver_2511531006 {
```

C. Gambar tersebut menunjukkan deklarasi variabel statis dan metode utama dalam kelas Driver. Baris ke-4 mendefinisikan sebuah Array bernama listMobil yang dipersiapkan untuk menampung maksimal 10 objek dari kelas Mobil_2511531006, sementara baris ke-5 mendeklarasikan variabel jumlah

untuk melacak jumlah objek yang benar-benar tersimpan. Penggunaan kata kunci static pada variabel-variabel ini memungkinkan mereka untuk diakses langsung oleh metode main (baris ke-7), yang berfungsi sebagai titik awal eksekusi program.

```
4       static Mobil_2511531006[] listMobil = new Mobil_2511531006[10];
5       static int jumlah = 0;
6
7       public static void main(String[] args) {
```

D. Gambar tersebut menunjukkan pemanggilan metode di dalam fungsi utama (main) untuk mengelola data objek. Pada baris 10 dan 11, metode tambah() dipanggil untuk memasukkan data mobil (seperti Avanza dan Civic) beserta spesifikasinya ke dalam struktur data atau array yang telah disiapkan sebelumnya. Selanjutnya, baris 13 mencetak judul ke konsol, dan baris 14 memanggil metode tampil() untuk mengeksekusi perintah penampilan seluruh data yang telah ditambahkan tersebut ke layar.

```
8
9           // Method Tambah Mobil
10          tambah("Avanza", 2020, 1300, 150000000L, "Toyota");
11          tambah("Civic", 2022, 1500, 500000000L, "Honda");
12
13          System.out.println("\n--- Data Awal ---");
14          tampil();
15
```

E. Gambar tersebut menunjukkan proses penghapusan data dari struktur penyimpanan dalam kelas Java. Pada baris 18, metode hapus(0) dipanggil untuk menghapus objek yang berada pada indeks ke-0 (data pertama) di dalam array. Setelah proses penghapusan selesai, baris 20 dan 21 digunakan untuk mencetak label serta memanggil kembali metode tampil() guna memperlihatkan kondisi data terbaru atau "Data Akhir" kepada pengguna.

```
16          // Method Hapus Mobil
17          System.out.println();
18          hapus(0); // Hapus index 0
19
20          System.out.println("\n--- Data Akhir ---");
21          tampil();
22      }
23
```

F. Gambar tersebut menunjukkan implementasi metode tambah yang berfungsi untuk memasukkan objek baru ke dalam array. Metode ini terlebih dahulu melakukan pengecekan kondisi pada baris 25 untuk memastikan bahwa variabel jumlah masih lebih kecil dari kapasitas maksimum array (listMobil.length). Jika kapasitas masih tersedia, program akan membuat objek Mobil_2511531006 baru pada indeks yang sesuai, mencetak pesan konfirmasi keberhasilan, dan melakukan increment (jumlah++) untuk memperbarui jumlah data yang tersimpan agar indeks berikutnya siap digunakan.

```
24      static void tambah(String n, int t, int c, long h, String m) {
25          if (jumlah < listMobil.length) {
26              listMobil[jumlah] = new Mobil_2511531006(n, t, c, h, m);
27              System.out.println("Berhasil tambah: " + n);
28              jumlah++;
29          }
30      }
```

G. Gambar tersebut menunjukkan implementasi metode hapus yang digunakan untuk menghapus elemen dari array berdasarkan indeks tertentu. Proses ini diawali dengan validasi indeks (baris 33) untuk memastikan indeks yang diminta tersedia dalam jangkauan data. Jika valid, program melakukan pergeseran elemen (*shifting*) menggunakan perulangan for (baris 35-37), di mana setiap elemen setelah indeks yang dihapus akan ditarik satu posisi ke depan untuk mengisi kekosongan. Terakhir, elemen di posisi paling ujung dikosongkan menjadi null (baris 38) dan variabel jumlah dikurangi satu (decrement) agar ukuran data tetap akurat.

```
31
32      static void hapus(int index) {
33          if (index >= 0 && index < jumlah) {
34              System.out.println("Menghapus: " + listMobil[index].getNama());
35              for (int i = index; i < jumlah - 1; i++) {
36                  listMobil[i] = listMobil[i + 1];
37              }
38              listMobil[jumlah - 1] = null;
39              jumlah--;
40          }
41      }
42
```

H. Gambar tersebut menunjukkan implementasi metode tampil yang berfungsi untuk mencetak seluruh daftar mobil yang tersimpan di dalam array ke layar. Metode ini menggunakan perulangan for (baris 44) yang berjalan dari indeks 0 hingga batas variabel jumlah. Di dalam perulangan tersebut, program

mencetak nomor urut yang dimulai dari angka 1 (i+1) dan kemudian memanggil metode displayInfo() dari setiap objek mobil pada indeks terkait (baris 46) untuk menampilkan rincian datanya secara lengkap. Secara keseluruhan, metode ini berperan sebagai fungsi pelaporan data yang terstruktur bagi pengguna.

```
43      static void tampil() {
44          for (int i = 0; i < jumlah; i++) {
45              System.out.print((i+1) + ". ");
46              listMobil[i].displayInfo();
47          }
48      }
49  }
```

G. Output

```
Console
<terminated> Driver_2511531006 [Java Application] C:\Users\Hype AMD\.p2\pool\p
Berhasil tambah: Avanza
Berhasil tambah: Civic

--- Data Awal ---
1. Avanza [Toyota] - Thn: 2020 - CC: 1300 - Harga: 150000000
2. Civic [Honda] - Thn: 2022 - CC: 1500 - Harga: 500000000

Menghapus: Avanza

--- Data Akhir ---
1. Civic [Honda] - Thn: 2022 - CC: 1500 - Harga: 500000000
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep pemrograman berorientasi objek dapat diterapkan dengan baik dalam pembuatan kelas Jam_2511531006. Kelas ini mampu merepresentasikan waktu dalam bentuk jam, menit, dan detik serta menyediakan berbagai operasi seperti validasi data, konversi ke detik, perbandingan waktu, dan operasi aritmatika.Selain itu, penggunaan atribut dengan modifier *private* serta penyediaan method seperti konstruktor, selector, dan operasi lainnya menunjukkan penerapan prinsip enkapsulasi yang baik. Program yang dibuat juga telah mampu menangani input pengguna serta melakukan pengolahan data waktu secara sistematis dan terstruktur.Dengan demikian, praktikum ini memberikan pemahaman yang lebih mendalam mengenai bagaimana merancang dan mengimplementasikan suatu kelas dalam pemrograman Java secara efektif.

3.2 Saran

Adapun saran yang dapat diberikan untuk pengembangan selanjutnya adalah sebagai berikut:Program dapat dikembangkan dengan menambahkan validasi yang lebih ketat, seperti penggunaan exception handling untuk menangani kesalahan input.

DAFTAR PUSTAKA

Herbert Schildt. 2018. *Java: The Complete Reference*. New York: McGraw-Hill Education.

Budi Raharjo. 2016. *Belajar Pemrograman Java untuk Pemula*. Bandung: Informatika.